LAPORAN PRAKTIKUM TEKNOLOGI IoT II

Modul 7 : Integrasi Backend Server Lokal (Docker & Mosquitto) dan Rule Engine (Node-RED)

Di susun oleh:

Nama : Mita Fitri Nadori

NPM : 25782102

Kelas : TRI-3D

Dosen Pengampu:

Dr. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGIREKAYASA INTERNET

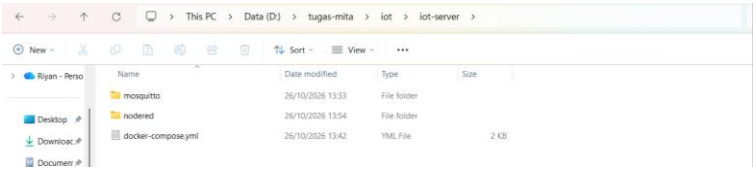
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

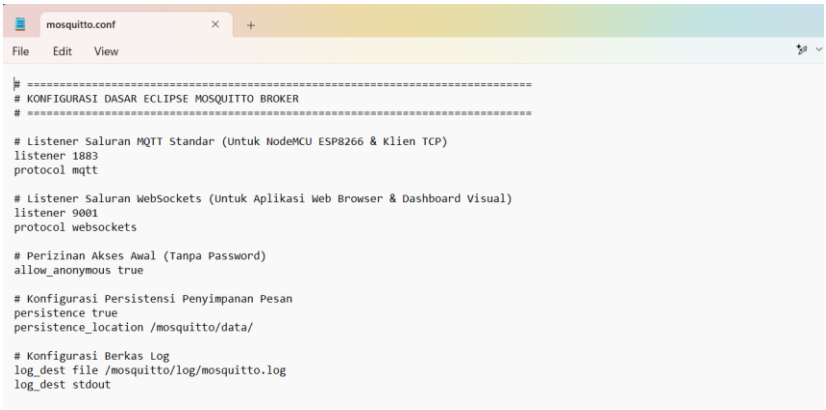
2026/2027

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

Di bagian ini saya membuat folder iot-server beserta folder mosquitto dan nodered untuk menyimpan file yang dibutuhkan selama praktikum.



File mosquitto.conf berisi pengaturan broker MQTT, yaitu port 1883 untuk MQTT dan port 9001 untuk WebSockets. Untuk pengujian awal, akses anonim masih diaktifkan.



```
# ================================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ================================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

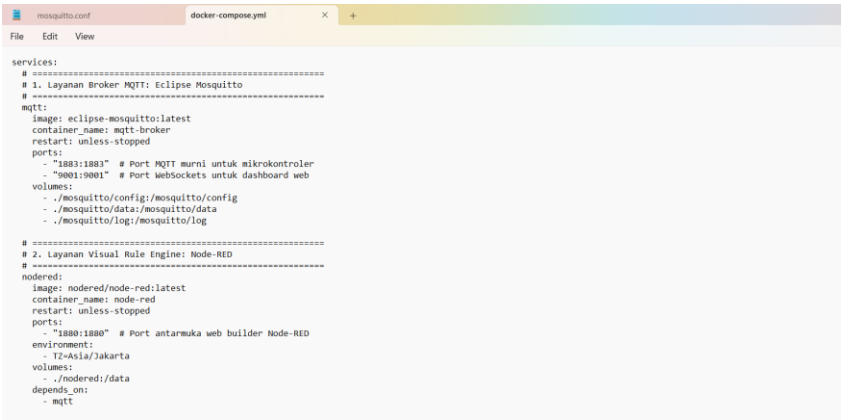
# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous true

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

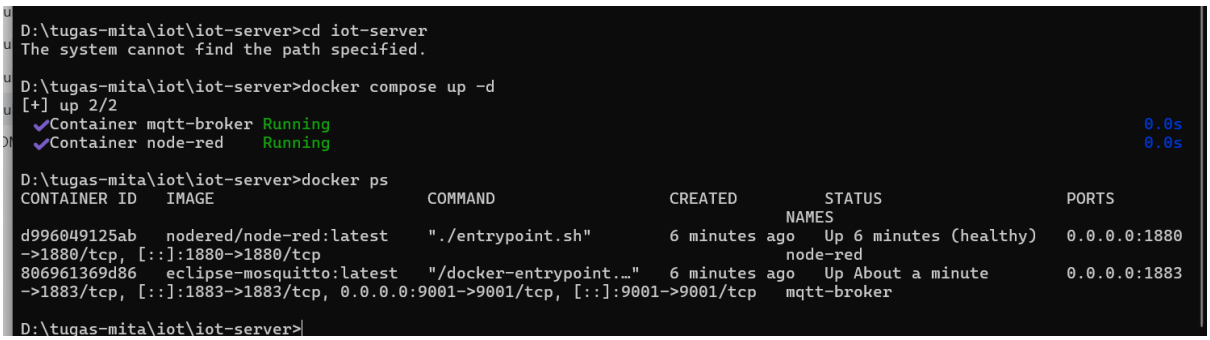
File docker-compose.yml digunakan untuk menjalankan Mosquitto dan Node-RED secara bersamaan. Port dan folder penyimpanan juga diatur melalui file ini.



```
services:
  # ==========================================================
  # 1. Layanan Broker MQTT: Eclipse Mosquitto
  # ==========================================================
  mqtt:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    restart: unless-stopped
    ports:
      - "1883:1883"  # Port MQTT murni untuk mikrokontroler
      - "9001:9001"  # Port WebSockets untuk dashboard web
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log

  # ==========================================================
  # 2. Layanan Visual Rule Engine: Node-RED
  # ==========================================================
  nodered:
    image: nodered/node-red:latest
    container_name: node-red
    restart: unless-stopped
    ports:
      - "1880:1880"  # Port antarmuka web builder Node-RED
    environment:
      - TZ=Asia/Jakarta
    volumes:
      - ./nodered:/data
    depends_on:
      - mqtt
```

Setelah konfigurasi selesai, saya menjalankan perintah docker compose up -d. Dari hasil terminal, container mqtt-broker dan node-red berhasil dimulai.



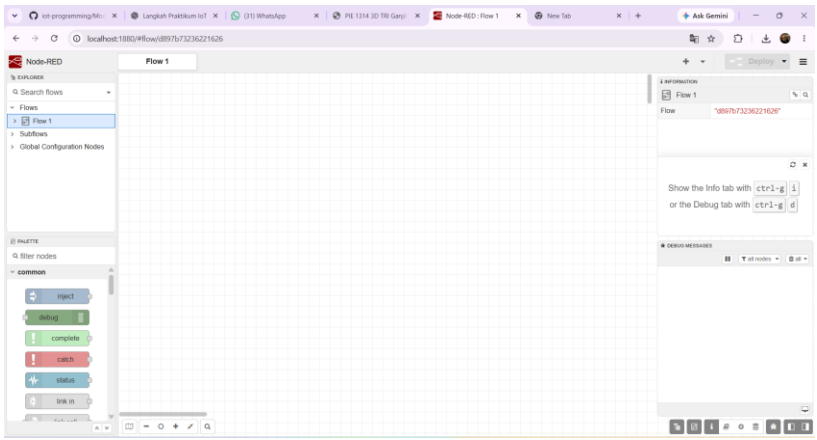
```
D:\tugas-mita\iot\iot-server>cd iot-server
The system cannot find the path specified.

D:\tugas-mita\iot\iot-server>docker compose up -d
[+] up 2/2
 ✔Container mqtt-broker Running                                                    0.0s
 ✔Container node-red    Running                                                    0.0s

D:\tugas-mita\iot\iot-server>docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED         STATUS                   PORTS
                                                                                  NAMES
d996049125ab   nodered/node-red:latest   "./entrypoint.sh"        6 minutes ago   Up 6 minutes (healthy)   0.0.0.0:1880
->1880/tcp, [::]:1880->1880/tcp                                                   node-red
806961369d86   eclipse-mosquitto:latest  "/docker-entrypoint.…"   6 minutes ago   Up About a minute        0.0.0.0:1883
->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp   mqtt-broker

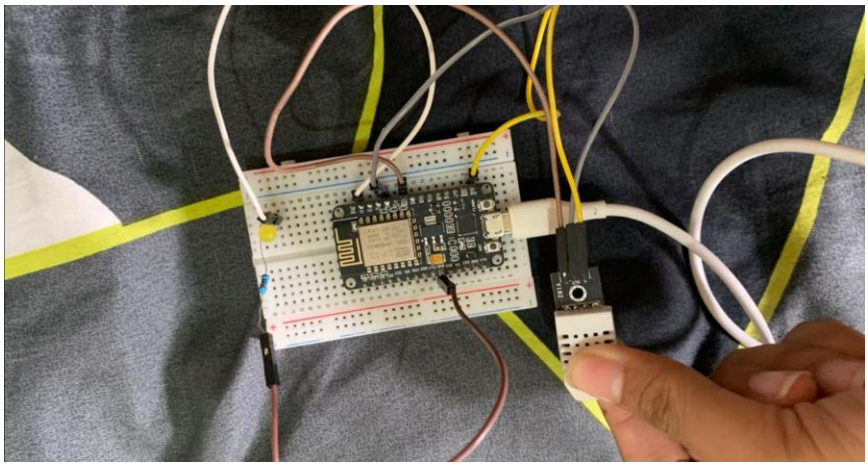
D:\tugas-mita\iot\iot-server>
```

Hasil pemeriksaan Docker menunjukkan kedua container sudah berjalan. Node-RED dapat diakses melalui port 1880, sedangkan Mosquitto menggunakan port 1883 dan 9001.

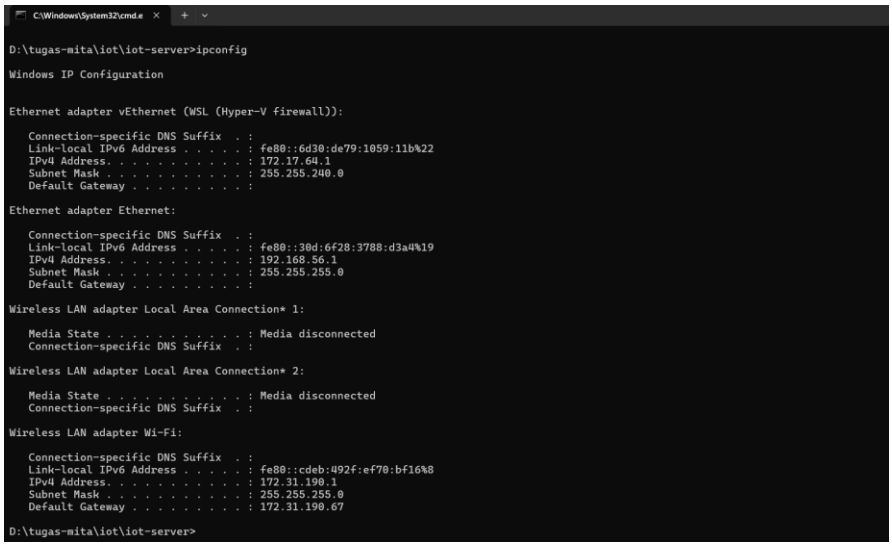


7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

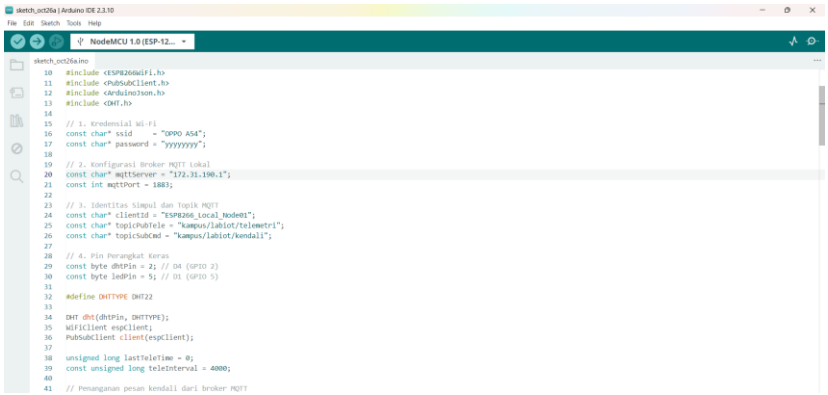
Rangkain yang dibuat:



Alamat IPv4 local laptop yang kita dapat



Kode dan Outputnya:



```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// 1. Kredensial Wi-Fi
const char* ssid     = "OPPO A54";
const char* password = "yyyyyyyy";

// 2. Konfigurasi Broker MQTT Lokal
const char* mqttServer = "172.31.190.1";
const int mqttPort = 1883;

// 3. Identitas Simpul dan Topik MQTT
const char* clientId = "ESP8266_Local_Node01";
const char* topicPubTele = "kampus/labiot/telemetri";
const char* topicSubCmd = "kampus/labiot/kendali";

// 4. Pin Perangkat Keras
const byte dhtPin = 2; // D4 (GPIO 2)
const byte ledPin = 5; // D1 (GPIO 5)

#define DHTTYPE DHT22

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTeleTime = 0;
const unsigned long teleInterval = 4000;

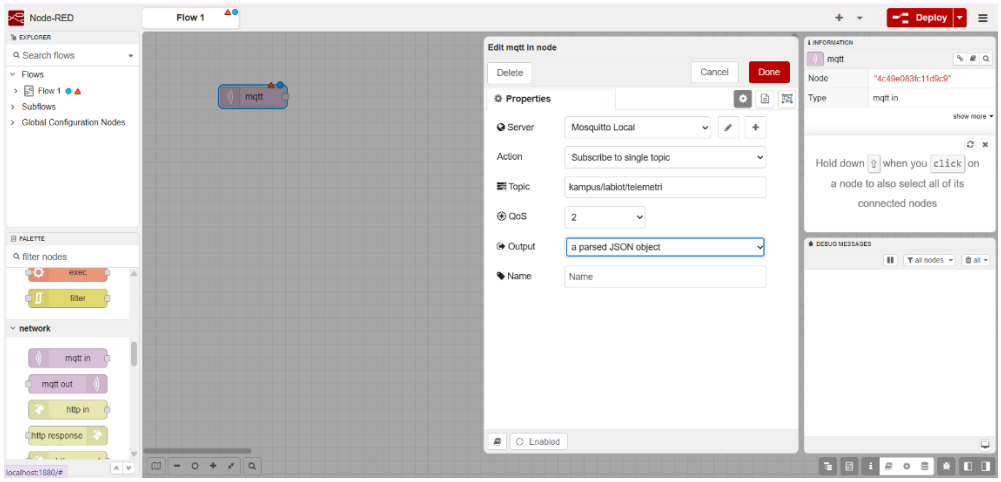
// Penanganan pesan kendali dari broker MQTT
```



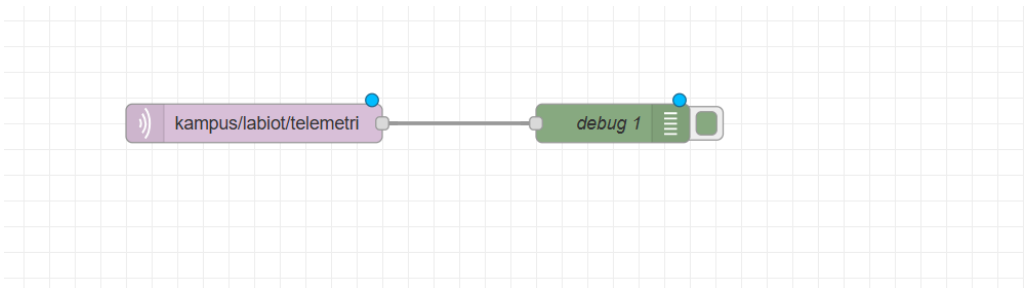
```
Output  Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')
New Line  115200 baud
14:30:34.477 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":30.00,"kelembapan":46.30,"status_led":0}
14:30:34.860 -> rlCiIcrCSCDb◆J◆pi◆◆◆CCCX◆cDb◆XCDnnC1nn2bbCpC5bC1r1pCnCD◆J◆CDnn◆n◆◆◆CbCDnn'1DD1'CCCDnn1'CCDnrCCDnbCD'CDrCDnbCD'CCCCC1'CCDn
14:30:34.975 -> [Wi-Fi] Menghubungkan ke OPPO A54
14:30:35.557 -> .............
14:30:42.312 -> [Wi-Fi] Berhasil Terhubung!
14:30:42.312 -> [Wi-Fi] IP NodeMCU: 172.31.190.90
14:30:42.318 -> [MQTT] Menghubungkan ke Broker Lokal (172.31.190.1)... Terhubung!
14:30:42.613 -> [MQTT] Berlangganan ke topik: kampus/labiot/kendali
14:30:42.613 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":30.00,"kelembapan":46.30,"status_led":0}
14:30:46.633 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":30.00,"kelembapan":46.20,"status_led":0}
14:30:50.628 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":30.00,"kelembapan":46.40,"status_led":0}
14:30:54.596 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":30.00,"kelembapan":46.50,"status_led":0}
```

7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

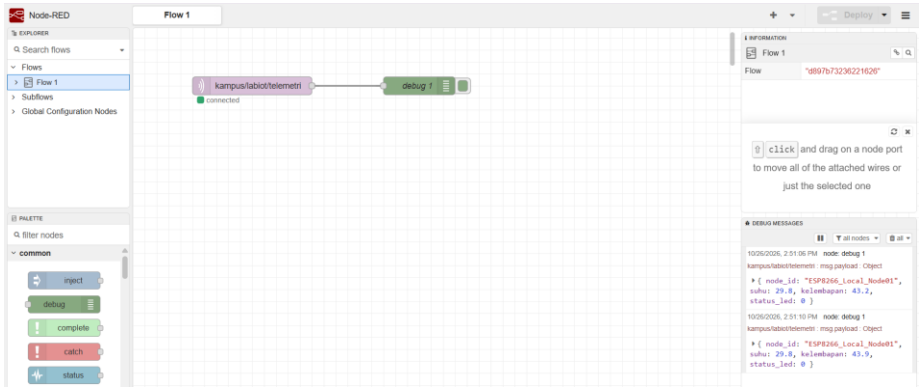
Halaman awal Node-RED menampilkan kanvas kerja yang masih kosong. Di kanvas ini saya menyusun node untuk menerima dan memeriksa data dari MQTT.



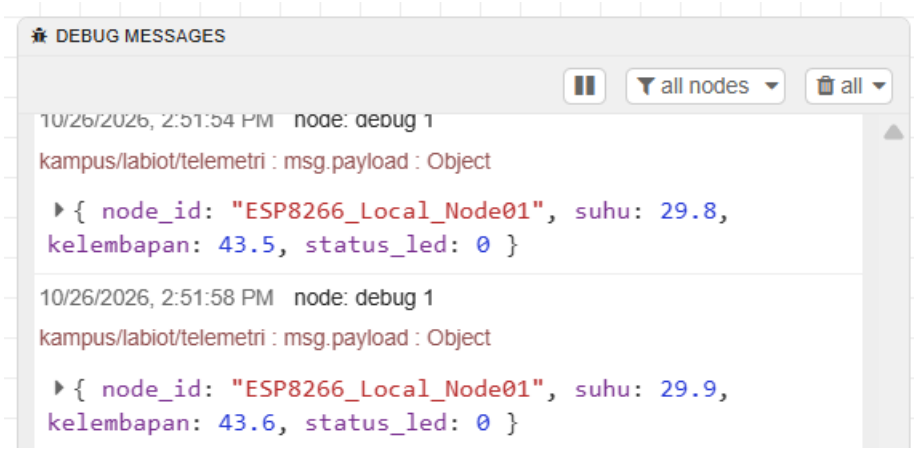
Pada pengaturan mqtt in, saya memilih broker Mosquitto Local dan mengisi topik kampus/labiot/telemetri. Output diatur menjadi objek JSON agar data sensor lebih mudah dibaca.



Node mqtt in dihubungkan ke node debug untuk menampilkan data yang masuk. Setelah alur di-deploy, pesan telemetri dapat dilihat melalui panel Debug.

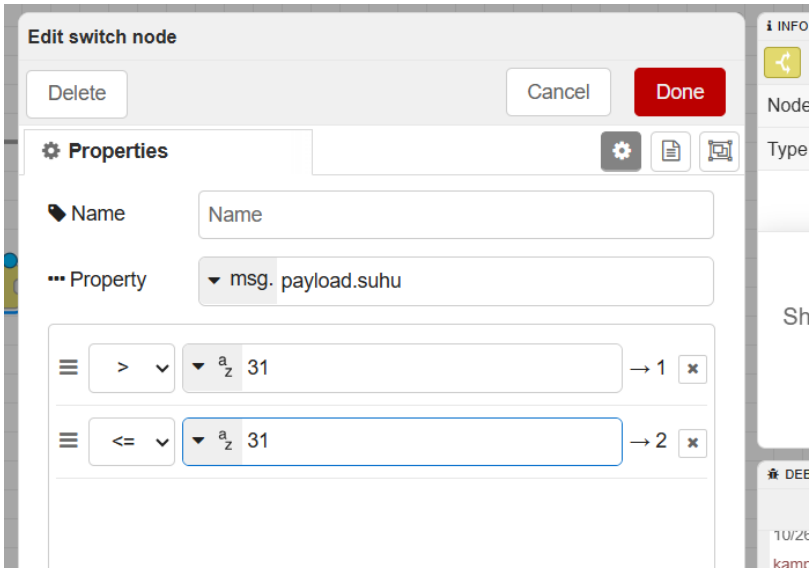


Panel Debug menampilkan data sensor seperti suhu, kelembapan, dan status LED. Hasil ini menunjukkan bahwa pesan dari ESP8266 sudah diterima oleh Node-RED.

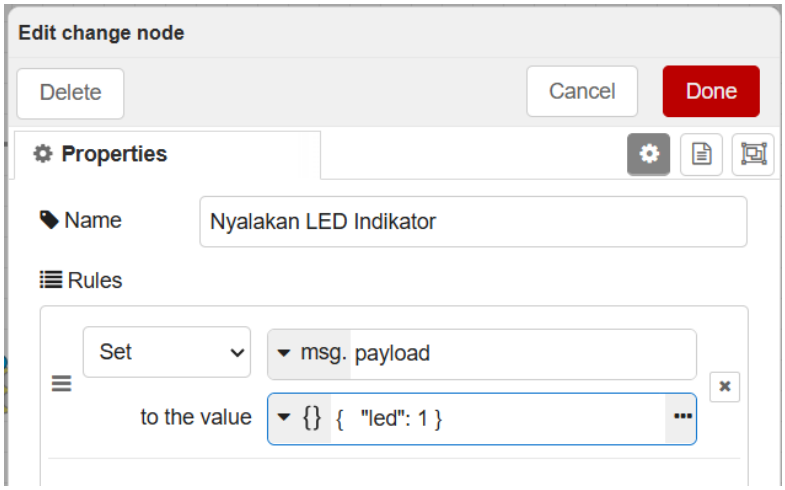


7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

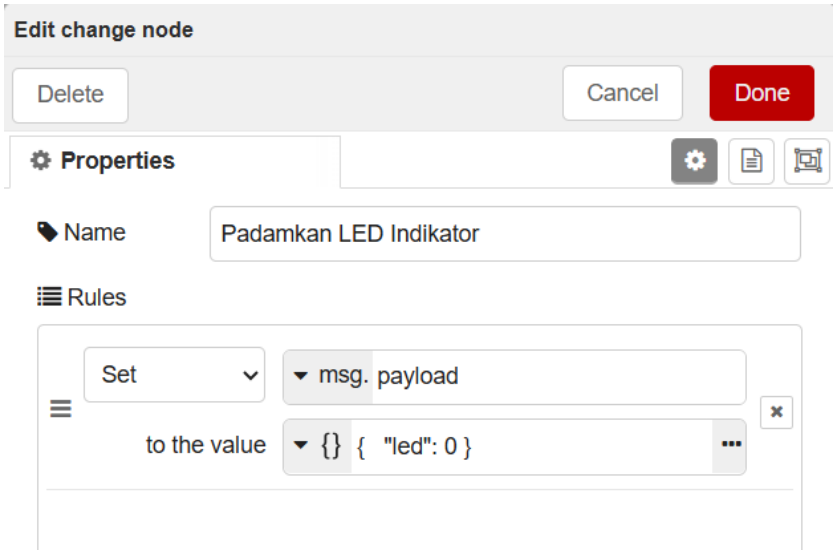
Membuat Node Percabangan Logika (switch):



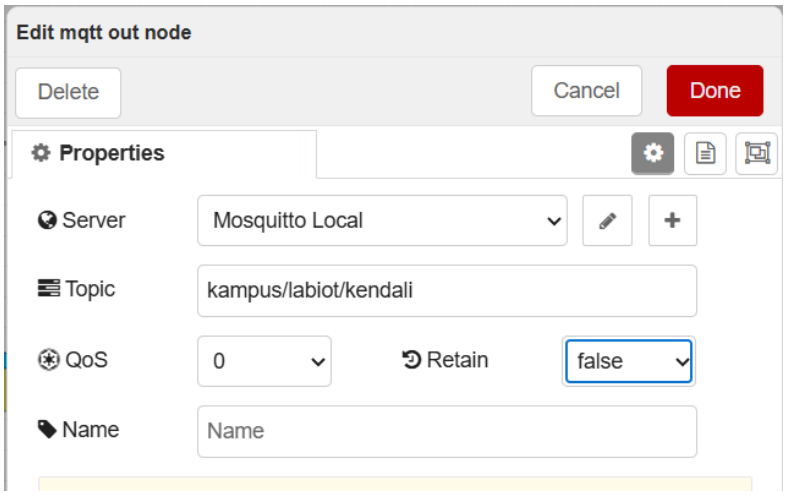
Membuat Node Transformasi Muatan Panas (change):

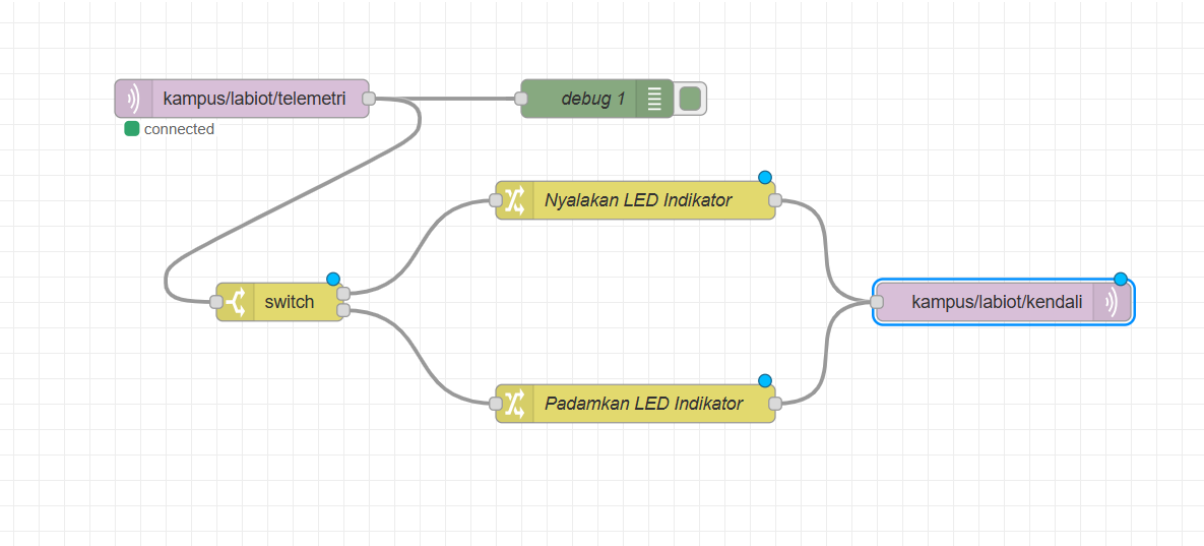


Membuat Node Transformasi Muatan Normal (change):

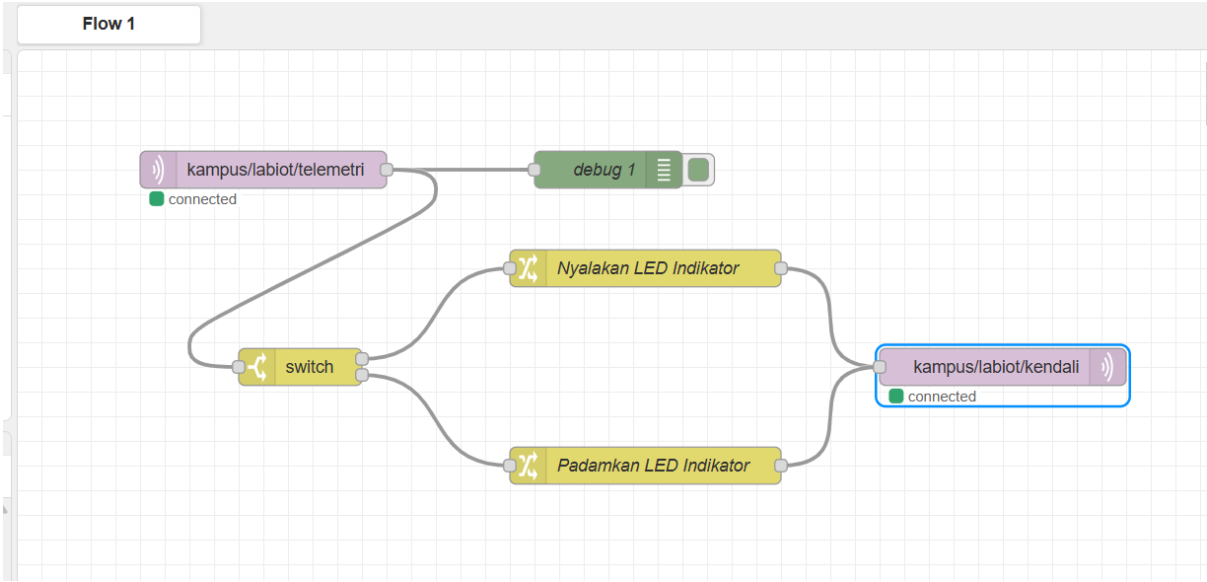


Menerbitkan Instruksi Balik ke Broker (mqtt out):

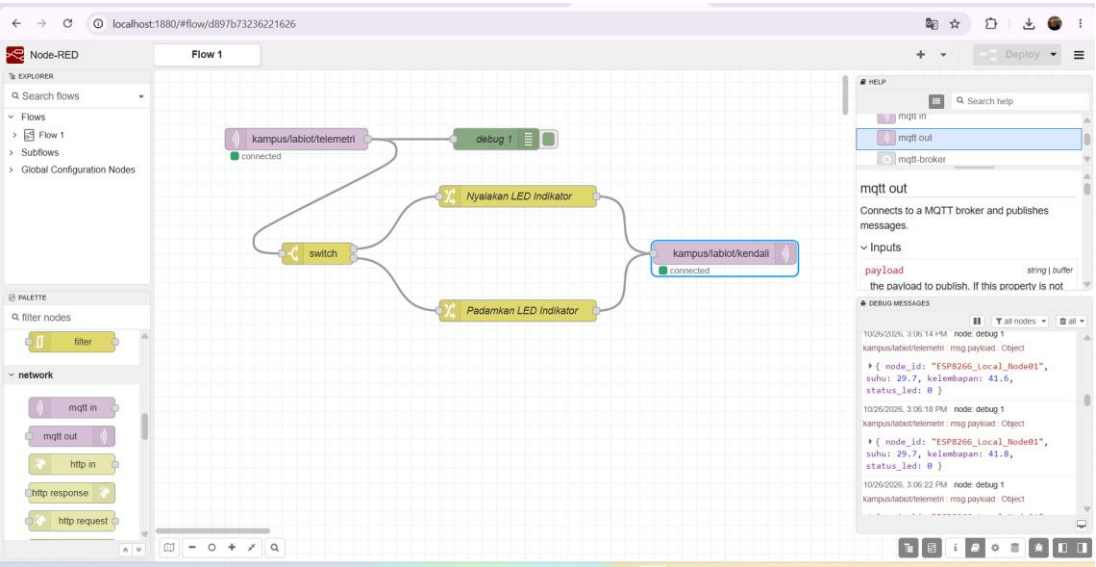




Menerapkan Perubahan (*Deploy*):



Pengujian Fisik Sistem Otomasi Tertutup:



7.6 Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan eksperimen teknis untuk melengkapi laporan praktikum Anda:

Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery)
Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks mqtt, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

Jawab: Karena localhost mengarah ke kontainer tempat Node-RED berjalan, bukan ke kontainer Mosquitto. Padahal, broker MQTT berada di kontainer yang berbeda. Akibatnya, Node-RED tidak bisa menemukan broker jika menggunakan localhost. Oleh karena itu, alamat server harus diisi mqtt agar Node-RED bisa terhubung ke broker melalui jaringan internal Docker.

ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

Jawab : Bridge Network merupakan jaringan internal yang menghubungkan kontainer agar bisa saling berkomunikasi. Sementara itu, Docker Embedded DNS berfungsi menerjemahkan nama layanan menjadi alamat IP kontainer yang sesuai. Jadi, ketika Node-RED menggunakan nama mqtt pada port 1883, Docker akan membantu mengarahkan koneksi tersebut ke layanan Mosquitto tanpa perlu memasukkan alamat IP secara manual.

Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):
Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

iii. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

Jawab : Karena alamat IP NodeMCU adalah 192.168.43.102, sedangkan alamat IP laptop adalah 192.168.1.15. Jika keduanya menggunakan subnet mask 255.255.255.0, kedua perangkat berada pada subnet yang berbeda. Akibatnya, NodeMCU tidak bisa langsung mengakses broker di laptop tanpa adanya konfigurasi routing yang sesuai.

iv. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

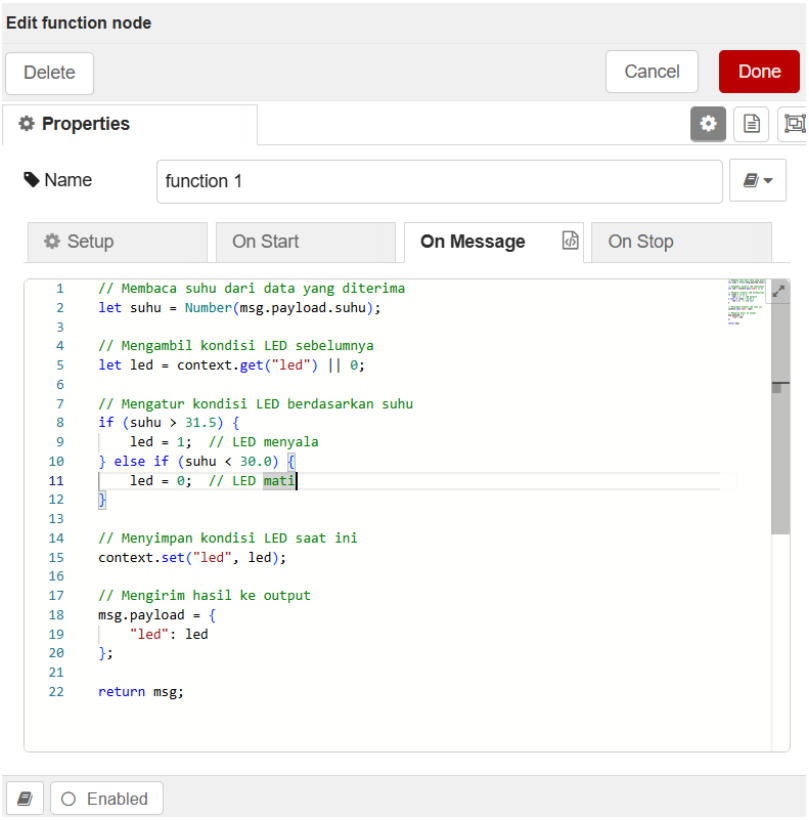
Jawab : Langkah pertama adalah menghubungkan laptop dan NodeMCU ke jaringan Wi-Fi atau hotspot yang sama. Setelah itu, periksa alamat IP laptop menggunakan perintah ipconfig. Masukkan alamat IPv4 laptop tersebut ke dalam program ESP8266 sebagai alamat broker MQTT. Terakhir, pastikan port 1883 diizinkan pada Windows Firewall agar koneksi dari NodeMCU tidak terblokir.

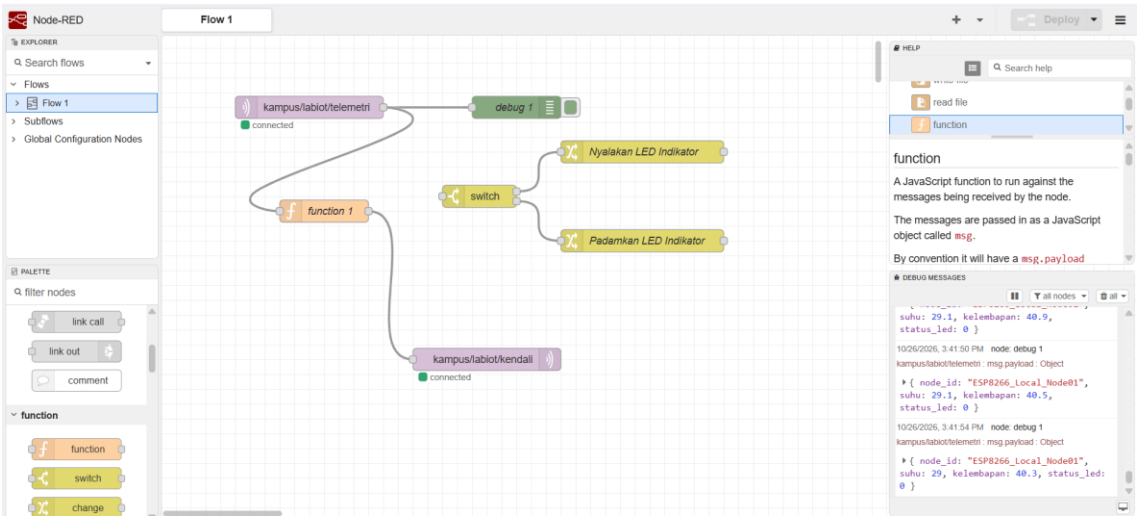
Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):
Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0°C(misal: 31.05°C → 30.95°C → 31.02°C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
Rancang modifikasi alur Node-RED menggunakan function node berbahasa JavaScript yang menerapkan logika Histeresis:

- LED baru menyala jika suhu > 31.5°C(msg.payload = {"led": 1}).
- LED baru padam jika suhu mendingin hingga < 30.0°C(msg.payload = {"led": 0}).

Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

Jawab :





Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):
Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

vii. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

Jawab : Jika baris volume ./nodered:/data dihapus, data konfigurasi dan flow Node-RED tidak lagi tersimpan pada folder proyek di laptop. Ketika kontainer dihentikan menggunakan docker compose down lalu dibuat kembali, flow yang sudah dibuat bisa hilang karena data sebelumnya berada di penyimpanan internal kontainer yang lama.

Hal yang sama berlaku untuk data Mosquitto jika volume ./mosquitto/data:/mosquitto/data dihapus. Data yang tersimpan di dalam kontainer lama tidak otomatis tersedia pada kontainer baru. Oleh karena itu, volume diperlukan agar data tetap tersimpan meskipun kontainer dihapus dan dibuat ulang.

viii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

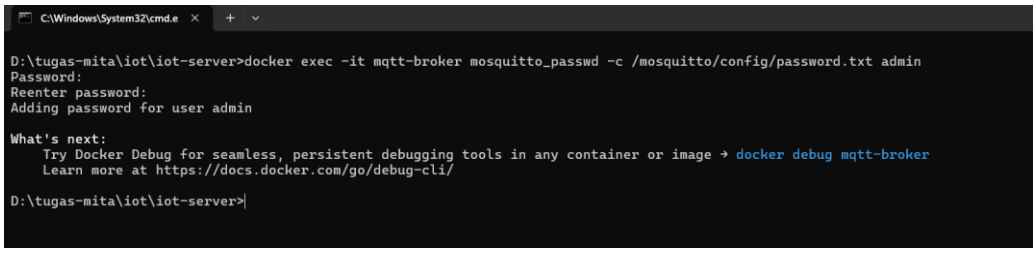
Jawab : Bind Mount adalah metode penyimpanan yang menghubungkan folder atau file tertentu di laptop dengan direktori di dalam kontainer. Contohnya, ./nodered:/data, yang berarti folder nodered pada proyek digunakan untuk menyimpan data Node-RED.

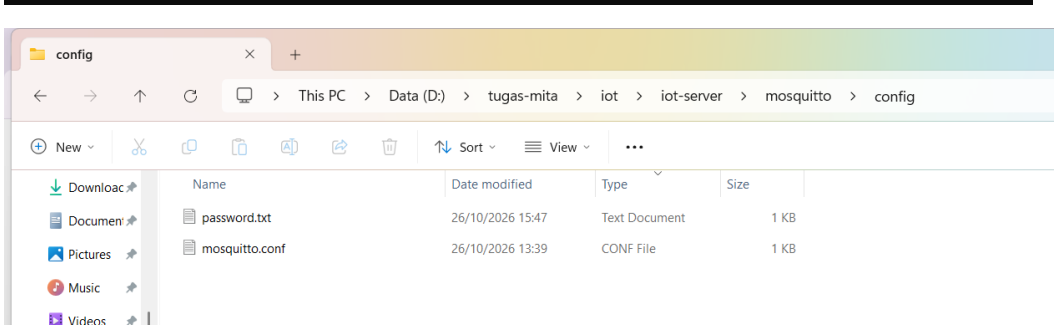
Sementara itu, Named Volume adalah penyimpanan yang dibuat dan dikelola oleh Docker dengan nama tertentu. Pengguna tidak perlu menentukan lokasi folder penyimpanannya secara langsung karena pengelolaannya dilakukan oleh Docker.

Jadi, perbedaan utamanya adalah Bind Mount menggunakan lokasi folder yang ditentukan pengguna, sedangkan Named Volume dikelola oleh Docker. Keduanya dapat digunakan untuk menjaga data tetap tersedia ketika kontainer dihentikan atau dibuat ulang.

7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:





Memperbarui Konfigurasi Kebijakan Mosquitto:

mosquitto.conf docker-compose.yml

File Edit View

```
# ================================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ================================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous false
password_file /mosquitto/config/password.txt

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

Memuat Ulang Layanan Broker:

```
D:\tugas-mita\iot\iot-server>docker restart mqtt-broker
mqtt-broker

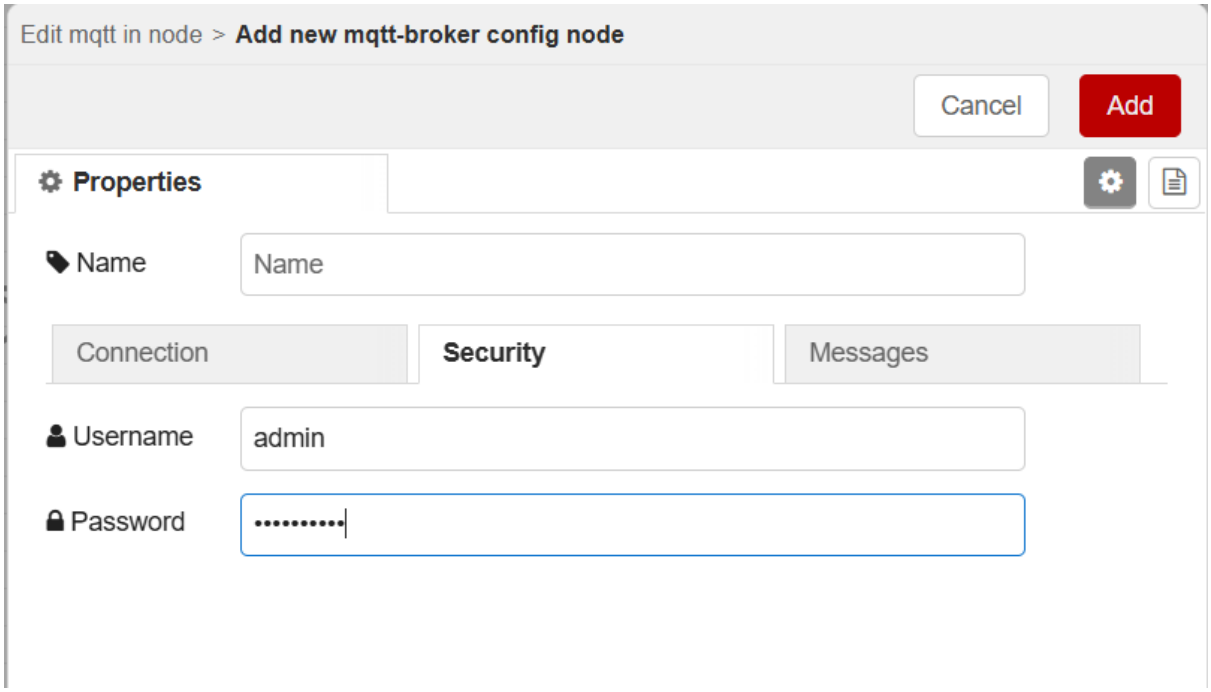
D:\tugas-mita\iot\iot-server>
```

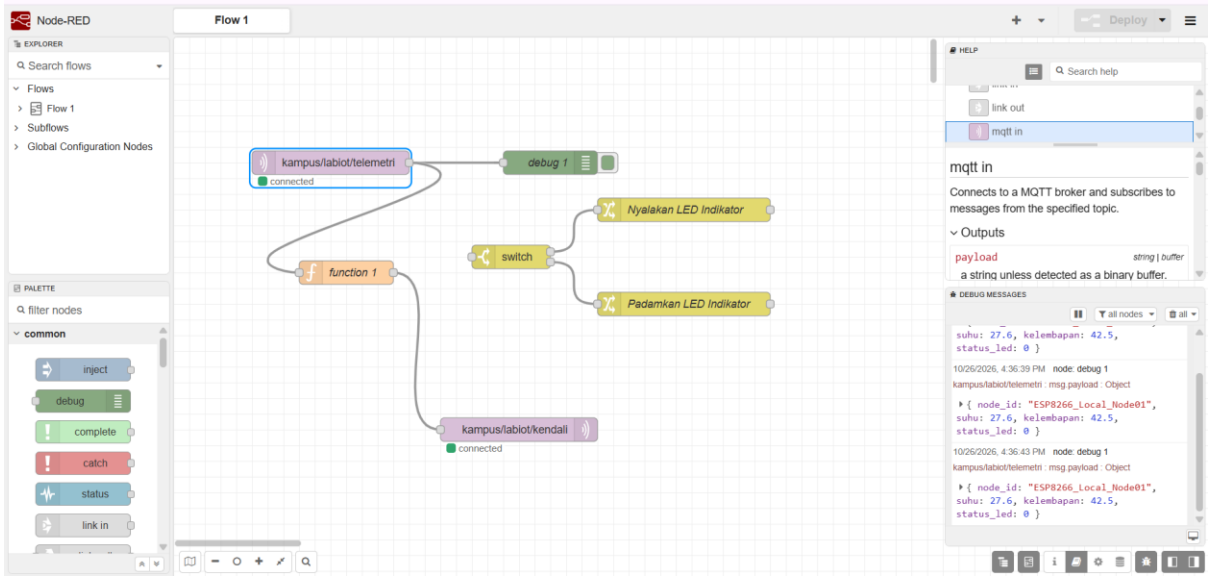
Analisis Dampak Kegagalan Sistem (Observasi Masalah):

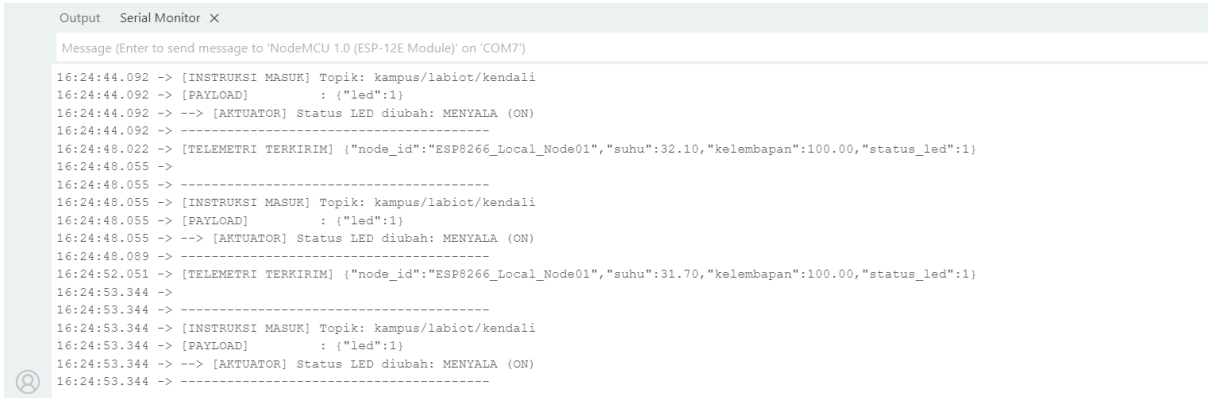
Setelah password diaktifkan, hasil output di monitor Arduino seperti ini

```
15:54:06.635 -> ----------------------------------------
15:54:10.551 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":29.10,"kelembapan":40.80,"status_led":0}
15:54:10.551 -> [MQTT] Menghubungkan ke Broker Lokal (172.31.190.1)... Gagal (rc=-4). Coba lagi dalam 4 detik...
15:54:29.553 -> [MQTT] Menghubungkan ke Broker Lokal (172.31.190.1)... Terhubung!
15:54:29.637 -> [MQTT] Berlangganan ke topik: kampus/labiot/kendali
15:54:29.677 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":29.10,"kelembapan":41.80,"status_led":0}
15:54:29.723 ->
```

Rekayasa Perbaikan Sistem (Penyelesaian Tugas):


Edit mqtt in node > Add new mqtt-broker config node
Cancel
Add
Properties
Name
Name
Connection
Security
Messages
Username
admin
Password


Node-RED
Flow 1
Deploy
Search flows
Flows
Flow 1
Subflows
Global Configuration Nodes
filter nodes
common
inject
debug
complete
catch
status
link in
kampus/labiot/telemetri
connected
debug 1
Nyalakan LED Indikator
switch
function 1
Padamkan LED Indikator
kampus/labiot/kendali
connected
Search help
link out
mqtt in
mqtt in
Connects to a MQTT broker and subscribes to messages from the specified topic.
Outputs
payload
string | buffer
a string unless detected as a binary buffer.
DEBUG MESSAGES


Output
Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')
16:24:44.092 -> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
16:24:44.092 -> [PAYLOAD] : {"led":1}
16:24:44.092 -> --> [AKTUATOR] Status LED diubah: MENYALA (ON)
16:24:44.092 -> ----------
16:24:48.022 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":32.10,"kelembapan":100.00,"status_led":1}
16:24:48.055 ->
16:24:48.055 -> ----------
16:24:48.055 -> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
16:24:48.055 -> [PAYLOAD] : {"led":1}
16:24:48.055 -> --> [AKTUATOR] Status LED diubah: MENYALA (ON)
16:24:48.089 -> ----------
16:24:52.051 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":31.70,"kelembapan":100.00,"status_led":1}
16:24:53.344 ->
16:24:53.344 -> ----------
16:24:53.344 -> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
16:24:53.344 -> [PAYLOAD] : {"led":1}
16:24:53.344 -> --> [AKTUATOR] Status LED diubah: MENYALA (ON)
16:24:53.344 -> ----------